

Report: PCIe LFSR

1. Architecture Overview

The block is a streaming, cut-through 8B CRC unit for a 2048-bit packet delivered over `NUM_CHUNKS` beats of `DATA_PATH_WIDTH` bits each — 4 beats × 512b, or 2 beats × 1024b, selected at compile time via the `DATA_PATH_WIDTH` macro in `crc_config.svh`. Framing uses `sop` (first beat), `eop` (last beat), and `valid` (per-beat qualifier); all other packet fields live in the packed `pkt_t` struct shared by `crc_pkg`.

The design supports two CRC datapath implementations: a standard, non-pipelined implementation and a pipelined implementation. The standard implementation is used for the baseline CRC calculation, while the pipelined implementation introduces an additional pipeline boundary within the CRC calculation to address the deeper combinational path associated with the 1024-bit configuration.

The standard implementation consists of two main blocks:

- **crc_calculator** — Purely combinational, computing `crc_out = f(crc_in, pkt_in.data, chunk_count)` using the precomputed `crc_matrix` parallel-CRC coefficient table. **Initially**, the matrix contained 2048 rows, with rows 111–0 filled with zeros. It was argued that these rows could be removed since they do not affect the XOR result and would only add area (112 rows × 8 CRC-byte elements). A row-index condition was therefore added to skip the removed entries. At this stage, `chunk_count` was converted to a runtime offset so that the same matrix could support both data widths. **However, this runtime offset would introduce comparison and matrix-selection logic**, since the offset affected both the payload-bound check and matrix row addressing for many CRC terms. **The implementation was subsequently updated** to use a constant `OFFSET = c × DATA_BYTES` for each `generate` branch, allowing these checks and matrix accesses to be resolved at build time.
- **crc_dut** — the framing FSM (IDLE/BUSY) plus the output register stage. Every cycle with `pkt_in.valid` asserted, `pkt_out.sop/eop/valid` and `pkt_out.data` are registered one cycle behind `pkt_in`. On all beats except the last, `pkt_out.data` is a straight passthrough of `pkt_in.data`. On the `eop` beat, the top 6 bytes of that beat are forced to 0 (reserved) and the next 8 bytes down are replaced with the completed CRC result; the remaining low-order bytes of that beat still pass through unmodified.

For the pipelined implementation, the CRC calculation is divided into two parallel partial-CRC calculations. Each calculation processes half of the input data width, and the two partial results are registered before being combined. The pipelined implementation therefore introduces an additional cycle of latency in the CRC path. The output/control logic is correspondingly delayed so that the packet data and FSM state remain aligned with the pipelined CRC result.

The running CRC (`crc_in`) is primed to `CRC_INIT` on the `sop` beat and cleared back to `CRC_INIT` on the `eop` beat, so back-to-back packets need no idle cycle between them. `chunk_count` tracks the current beat position and feeds the `crc_matrix` row offset. Correct operation assumes upstream framing delivers exactly `NUM_CHUNKS` valid beats between `sop` and `eop`. An early or missing `eop` can therefore desynchronize the row offset for the remainder of the packet and the subsequent packet. This assumption is called out directly in the RTL.

1.1 Diagrams



Figure 1: *Packet Struct*

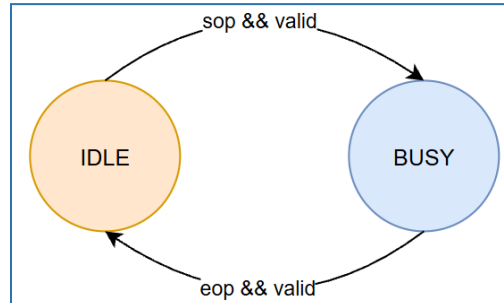


Figure 2: *State Diagram*

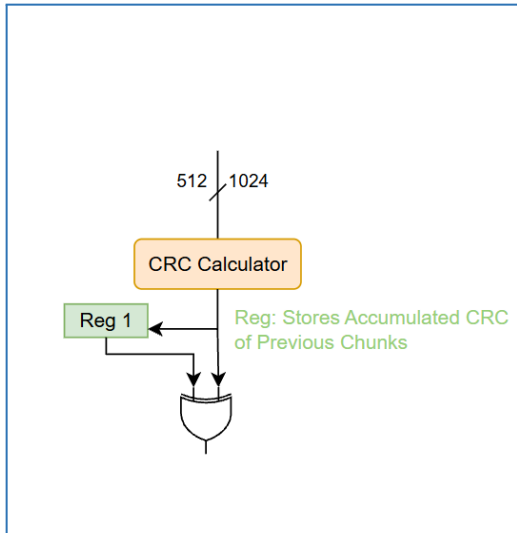


Figure 3: *Standard Block Diagram*

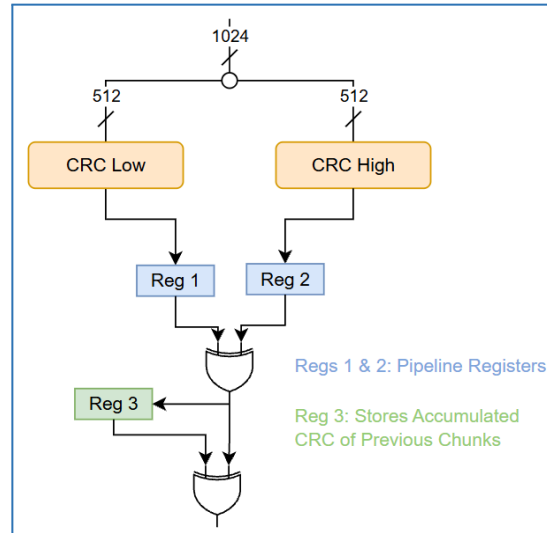


Figure 4: *Pipelined Block Diagram*

1.2 Pipeline Placement

In the standard implementation, the complete `crc_matrix` XOR/AND reduction inside `crc_calculator` is combinational between the CRC state register and the output register. There is no register boundary within the CRC calculation itself. Consequently, a given input beat is processed completely within one clock cycle and the corresponding output beat is produced one cycle later. The standard implementation therefore has a single registered boundary from `pkt_in` to `pkt_out`, with the main CRC combinational path running through.

The running CRC for the next beat is derived from the completed CRC result of the current beat, so the full CRC reduction must complete within one clock period on every valid beat.

The pipelined implementation modifies this structure by introducing a register boundary within the CRC calculation. For the 1024-bit configuration, the input is divided into two 512-bit portions. Each portion is processed by an independent CRC calculation in parallel, and the resulting partial CRC values are

registered. The registered partial results are then combined in the following cycle to form the CRC result for the corresponding input beat.

The pipelined implementation therefore has an additional cycle of latency relative to the standard implementation. Once the pipeline is filled, however, the CRC calculation for the next input beat can proceed in parallel with the combination of the partial results from the preceding beat. Thus, the additional latency does not reduce the steady-state throughput.

Because the CRC result is delayed by the additional pipeline stage, the pipelined `crc_dut` also registers the input packet and corresponding FSM state. The output logic operates on these delayed values so that `pkt_out`, the FSM state, and the CRC result all refer to the same input beat. The CRC calculation itself continues to operate on the current input beat, allowing consecutive beats to be processed without introducing a throughput gap.

1.3 Timing Considerations

The standard and pipelined implementations have different timing characteristics.

For the standard implementation:

- The 1024-bit configuration (`DATA_BYTES = 128`) has approximately twice the number of contribution terms per beat as the 512-bit configuration (`DATA_BYTES = 64`). Consequently, its CRC reduction represents a substantially larger combinational network and is expected to be more timing-critical at the same target frequency.
- The `crc_in → crc_calculator → crc_out → crc_in` recurrence requires the complete CRC reduction for each beat to finish within one clock period. There is no pipeline boundary inside the reduction to distribute this delay across multiple cycles.
- `chunk_count` is used to generate the `start_row` offset for `crc_matrix` addressing. This offset calculation and the resulting matrix indexing are part of the per-beat combinational CRC calculation.

For the pipelined implementation:

- The 1024-bit input is divided into two 512-bit portions that are processed in parallel.
- Each partial CRC calculation is registered, creating a pipeline boundary before the two partial results are combined.
- The manually defined split does not itself reduce the theoretical XOR depth from 10 levels to 5 levels. Each 512-input reduction can still have a theoretical depth of approximately 9 levels for a balanced 2-input XOR tree. The purpose of the split is instead to introduce a pipeline boundary while keeping the RTL relatively simple and allowing synthesis greater freedom to optimize the implementation.
- The pipelined implementation therefore trades one additional cycle of latency for a potential improvement in the achievable clock frequency.

In both implementations, bubble cycles (`valid = 0`) do not update the CRC values. State and CRC values are therefore retained until the next valid beat.

1.4 512-bit vs. 1024-bit Differences

Both configurations use the same CRC matrix coefficient table and the same basic CRC algorithm. The primary difference is the number of input bytes processed per beat, which is determined by the `DATA_PATH_WIDTH` configuration.

Parameter	512-bit mode	1024-bit mode
<code>DATA_BYTES</code>	64	128
<code>NUM_CHUNKS</code> (beats/packet)	4	2
Input data per cycle	512 bits	1024 bits
Pipelined CRC option	—	Used to reduce timing pressure from the wider CRC reduction
Standard input-to-output latency	1 cycle	1 cycle
Pipelined input-to-output latency	—	2 cycles

The 1024-bit configuration processes twice as much data per cycle and therefore requires only two beats to process the 2048-bit packet, compared with four beats in the 512-bit configuration. This increases the amount of combinational CRC work performed per cycle and makes the timing of the CRC reduction more challenging.

The pipelined implementation addresses this timing challenge by dividing the 1024-bit CRC calculation into two parallel 512-bit partial calculations and registering their results. This increases the latency by one cycle but preserves the ability to accept and process consecutive input beats once the pipeline is filled.

The output flit framing remains width-mode-independent. On the eop beat, the same CRC insertion scheme is applied relative to `DATA_BYTES`: the top 6 bytes are reserved and forced to zero, the next 8 bytes contain the CRC, and the remaining bytes are passed through unchanged.

No separate CRC coefficient matrix is required for the different data widths. The same 2048-row matrix is used in all configurations; only the number of rows consumed per beat and the `start_row` addressing change with `DATA_BYTES`.

2 Understanding and Implementation of CRC Pipelining

The standard CRC implementation contains a large XOR reduction network. For an XOR tree with N inputs and a fan-in of F , the theoretical combinational depth is $\text{ceil}(\log_F(N))$. For a 2-input XOR tree with 1024 inputs, this corresponds to 10 XOR levels. Such a deep combinational path can become the critical path and limit the achievable clock frequency.

Initially, splitting the XOR tree by depth, placing a pipeline register approximately halfway through the tree was considered. For a 1024-input, 2-input XOR tree, this would ideally result in two pipeline stages of approximately 5 XOR levels each. However, implementing this directly in RTL would require explicitly constructing the XOR tree and registering its intermediate nodes, increasing RTL complexity and making the implementation more dependent on the specific tree structure.

Instead, the pipelined implementation divides the 1024-bit input into two 512-bit halves. Each half is reduced independently, and the resulting partial CRC values are registered before being combined in the following cycle. This approach does not inherently reduce the theoretical XOR depth from 10 levels to 5 levels; each 512-input reduction can still have a theoretical depth of approximately 9 levels for a balanced 2-input XOR tree.

The main objective of this approach is therefore not to manually create a balanced two-stage XOR tree, but to introduce a pipeline boundary while keeping the RTL relatively simple. The added register boundary provides the synthesis tool with an opportunity to optimize the implementation and, where retiming is enabled and permitted by the timing constraints, redistribute registers relative to the combinational logic.

The pipelined implementation consequently introduces one additional cycle of latency compared with the standard implementation. However, the two partial CRC calculations operate in parallel, and while the registered results from one input beat are being combined, the next input beat can already enter the partial CRC calculations. Once the pipeline is filled, consecutive beats can therefore continue to be processed at one beat per cycle.

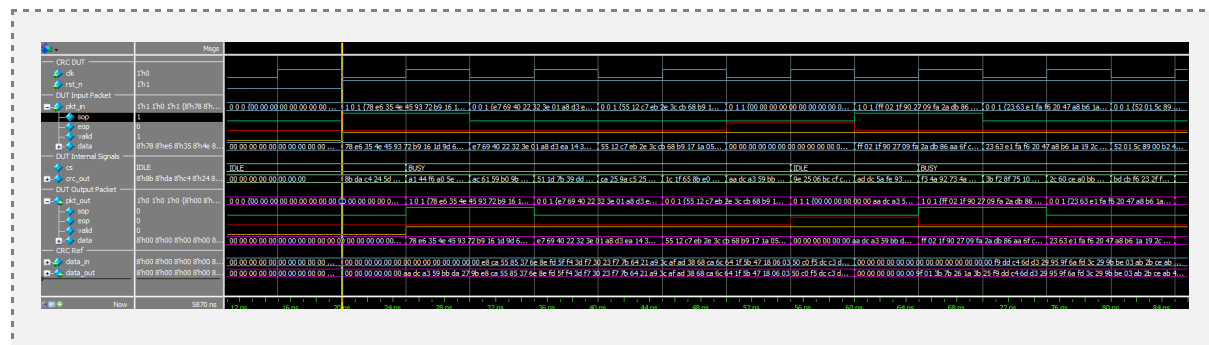
This approach was selected as a practical compromise between reducing the combinational critical path, maintaining manageable RTL complexity, and giving the synthesis tool flexibility to optimize the final implementation. The actual effectiveness of register retiming and the resulting critical-path improvement depend on the synthesis tool, retiming settings, and timing constraints.

3. Simulation Results

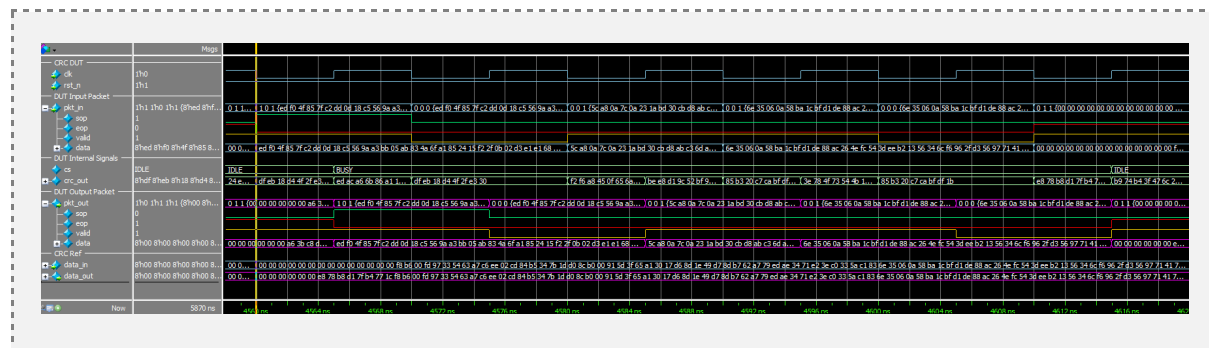
The design was verified using a SystemVerilog testbench containing several complementary test scenarios. Following reset, the first test applies 100 randomly generated packets back-to-back, with no idle cycles between packets, to verify continuous packet processing and correct CRC accumulation across consecutive packets. The second test introduces idle cycles between randomly generated packets to verify that the design correctly handles gaps in the input stream without losing or corrupting its CRC state. The third and fourth tests apply deterministic packets containing all-zero and all-one data patterns, respectively, providing simple corner cases that exercise the CRC logic with highly uniform input data. The fifth test applies 20 additional random packets with randomly inserted bubble cycles (`valid = 0`) to verify that the design correctly preserves its state when no valid input is present. For each test, the generated output is compared against a reference CRC calculation, with the comparison accounting for the pipeline latency of the pipelined implementation. The testbench maintains success and error counters, and all checks are allowed to complete before the final results are reported.

3.1 512-bit Data Path

Consecutive Valid Chunks

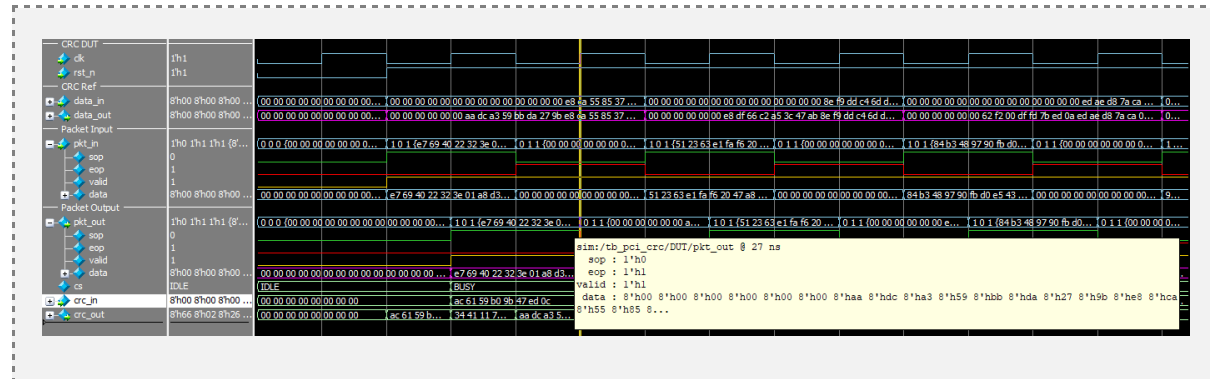


Non-Consecutive Valid Chunks

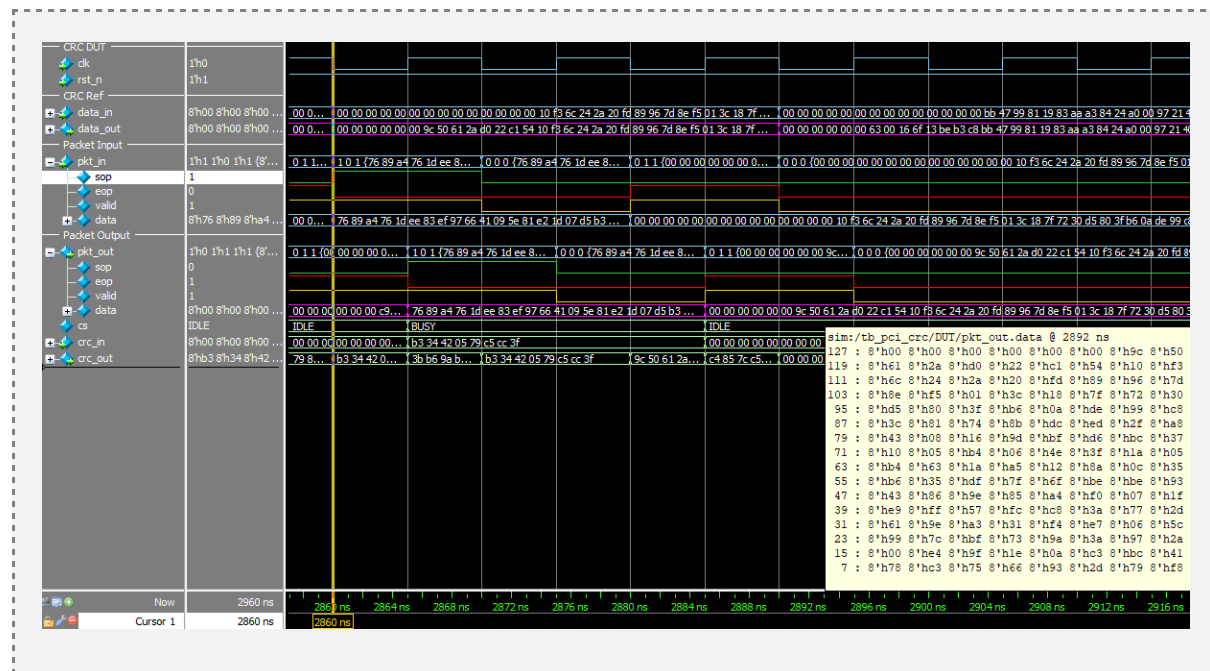


3.2 1024-bit Data Path

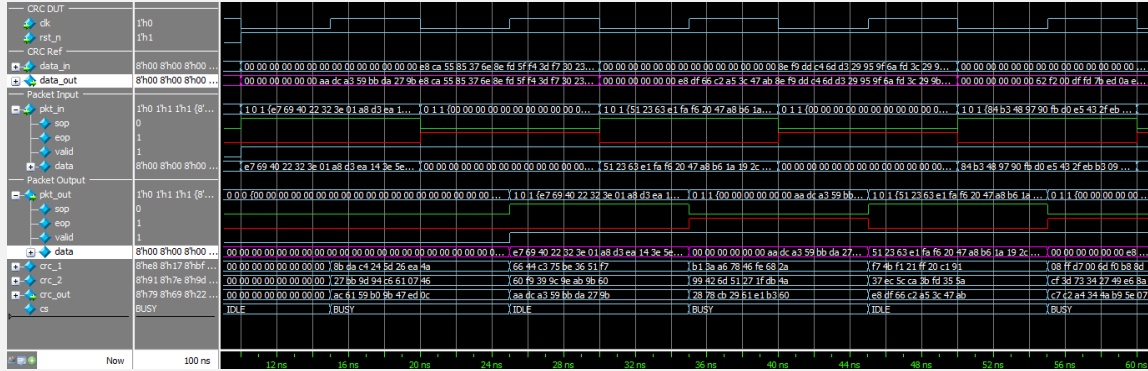
Standard - Consecutive Valid Chunks



Standard - Non-Consecutive Valid Chunks



Pipelined - Consecutive Valid Chunks



Pipelined - Non-Consecutive Valid Chunks

